

Textual Analysis in Finance

Week 4: Transformers and Language Models

Tri, Minh Phan

Faculty of Business and Economics
University of Basel

Where are we?



Outline

1. Self-attention
2. Transformer
3. Language models
4. Tokenization
5. Language models in financial research
6. Coming next

Self-attention

Previously on ELMo

ELMo ...

- ▶ ... equips static embeddings with context information
- ▶ ... produces contextualized embeddings

but, it ...

- ▶ ... suffers from the **gradient-vanishing issue**
- ▶ ... is computationally expensive

How to solve these issues?

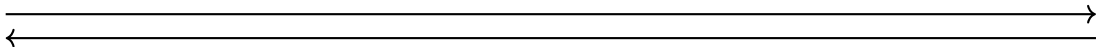
“Attention Is All You Need”

- ▶ Vaswani et al. [2017] at Google Brain & Google Research
- ▶ Introduced the *self-attention*: no recurrence, no convolution—only self-attention
- ▶ Revolutionize the way human models text data!
- ▶ $\approx 216,000$ citations (Google Scholar, on 29th, Jan 2026)
The most cited paper in finance, Fama [1970], has $\approx 45,000$ citations
- ▶ The foundation of BERT, GPT, and modern large language models

Intuition - General picture

ELMo:

The company reported strong cash flows ... however, due to regulatory uncertainty ...



Do we really need to do that?

*The company reported **strong cash flows** ... however, due to **regulatory uncertainty** ...*

1. Read the sentence all at once
2. Identify keywords
3. Extract features by putting high attention (weights) into keywords

Intuition - The **attention** part

Identifying keywords = Searching!

transformers, deep learning

+ Create

Visualizing transformers and attention | Talk for TNG Big Tech Day '24
1M views • 1 year ago
Grant Sanderson
An overview of transforms, as used in LLMs, and the attention mechanism within them. Based on the 3blue1brown deep learning ...
+ Summary This talk visually explains Transformer models, based on 3Blue1Brown's deep learning series. The presenter... -

Transformers Explained - How transformers work
3.3M views • 3 years ago
The Engineering Mindset
How transformers work Skillshare: <https://skl.sh/theengineeringmindset05221> The first 1000 people to use the link or my code ...
4K
3 chapters Intro | What are transformers | Basic calculations -

How Power Transformers work ? | Epic 3D Animation #transformers
1.1M views • 1 year ago
The science works
transformers #transformer #induction Power transformers are crucial for ensuring a steady and safe supply of electricity to homes ...
+ Summary This 3D animation explores the inner workings of power transformers. It details the core components and their... -

Intuition - The **attention** part

transformers, deep learning

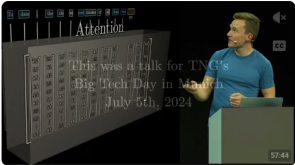
Query (Q)

X

Q

🔊

+ Create



This was a talk for TNG's Big Tech Day in March July 5th, 2024

57:44

Visualizing transformers and attention | Talk for TNG Big Tech Day '24

1M views • 1 year ago

 Grant Sanderson

An overview of transforms, as used in LLMs, and the attention mechanism within them. Based on the 3blue1brown deep learning ...

+

 Summary This talk visually explains Transformer models, based on 3Blue1Brown's deep learning series. The presenter ...



ELECTRICAL TRANSFORMERS EXPLAINED

16:33

Transformers Explained - How transformers work

3.3M views • 3 years ago

 The Engineering Mindset

How transformers work Skillshare: <https://skl.st/theengineeringmindset05221> The first 1000 people to use the link or my code ...

4K

 3 chapters

 Intro | What are transformers | Basic calculations



TRANSFORMERS

21:36

How Power Transformers work ? | Epic 3D Animation #transformers

1.1M views • 1 year ago

 The science works

transformers #transformer #induction Power transformers are crucial for ensuring a steady and safe supply of electricity to homes ...

+

 Summary This 3D animation explores the inner workings of power transformers. It details the core components and their ...

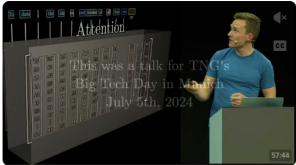
Intuition - The **attention** part

transformers, deep learning

Query (Q)

+

Create



Visualizing transformers and attention | Talk for TNG Big Tech Day '24

1M views • 1 year ago

Grant Sanderson

An overview of transforms, as used in LLMs, and the attention mechanism within them. Based on the 3blue1brown deep learning ...

Summary This talk visually explains Transformer models, based on 3Blue1Brown's deep learning series. The presenter...

Key (K_1)



Transformers Explained - How transformers work

3.3M views • 3 years ago

The Engineering Mindset

How transformers work Skillshare: <https://skl.sh/theengineeringmindset05221> The first 1000 people to use the link or my code ...

4K

3 chapters Intro | What are transformers | Basic calculations

Key (K_2)



How Power Transformers work ? | Epic 3D Animation #transformers

1.1M views • 1 year ago

The science works

transformers #transformer #induction Power transformers are crucial for ensuring a steady and safe supply of electricity to homes ...

Summary This 3D animation explores the inner workings of power transformers. It details the core components and their...

Key (K_3)

Intuition - The **attention** part

transformers, deep learning

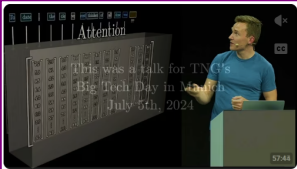
Query (Q)

×

Q

🔊

+ Create



This was a talk for TNG's Big Tech Day in Munich July 5th, 2024

57:44

Visualizing transformers and attention | Talk for TNG Big Tech Day '24

17K views • 1 year ago

Grant Sanderson

An overview of transforms, as used in LLMs, and the attention mechanism within them. Based on the 3blue1brown deep learning ...

Summary This talk visually explains Transformer models, based on 3Blue1Brown's deep learning series. The presenter...



ELECTRICAL TRANSFORMERS EXPLAINED

16:33

Transformers Explained - How transformers work

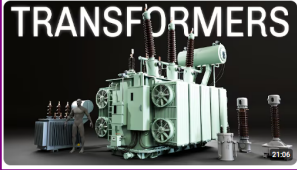
3.5M views • 5 years ago

The Engineering Mindset

How transformers work Skillshare: <https://skl.sh/theengineeringmindset05221> The first 1000 people to use the link or my code ...

4K

3 chapters Intro | What are transformers | Basic calculations



TRANSFORMERS

21:06

How Power Transformers work ? | Epic 3D Animation #transformers

1.1M views • 1 year ago

The science works

transformers #transformer #induction Power transformers are crucial for ensuring a steady and safe supply of electricity to homes ...

Summary This 3D animation explores the inner workings of power transformers. It details the core components and their...

Key (K_1)

Value (V_1)

Key (K_2)

Value (V_2)

Key (K_3)

Value (V_3)

Intuition - The **attention** part

- ▶ **Searching analogy:** Find the video whose **key** is most similar to the query

Intuition - The **attention** part

- ▶ **Searching analogy:** Find the video whose **key** is most similar to the query
- ▶ Let
 - ▶ Query: Q , a d -dimensional vector
 - ▶ Video t 's Key: K_t , d -dimensional
 - ▶ Video t 's Value: V_t , d -dimensional

The similarity between the query and video t is computed as:

$$\text{similarity}_t = \frac{QK_t^\top}{\sqrt{d}}$$

Intuition - The **attention** part

- ▶ **Searching analogy:** Find the video whose **key** is most similar to the query
- ▶ Let
 - ▶ Query: Q , a d -dimensional vector
 - ▶ Video t 's Key: K_t , d -dimensional
 - ▶ Video t 's Value: V_t , d -dimensional

The similarity between the query and video t is computed as:

$$\text{similarity}_t = \frac{QK_t^\top}{\sqrt{d}}$$

- ▶ The final output of the search (weighted sum of values) is:

$$\text{search_value} = \sum_{t=1}^T \text{softmax}(\text{similarity}_t) V_t = \sum_{t=1}^T \text{softmax}\left(\frac{QK_t^\top}{\sqrt{d}}\right) V_t$$

Intuition - The **self** part

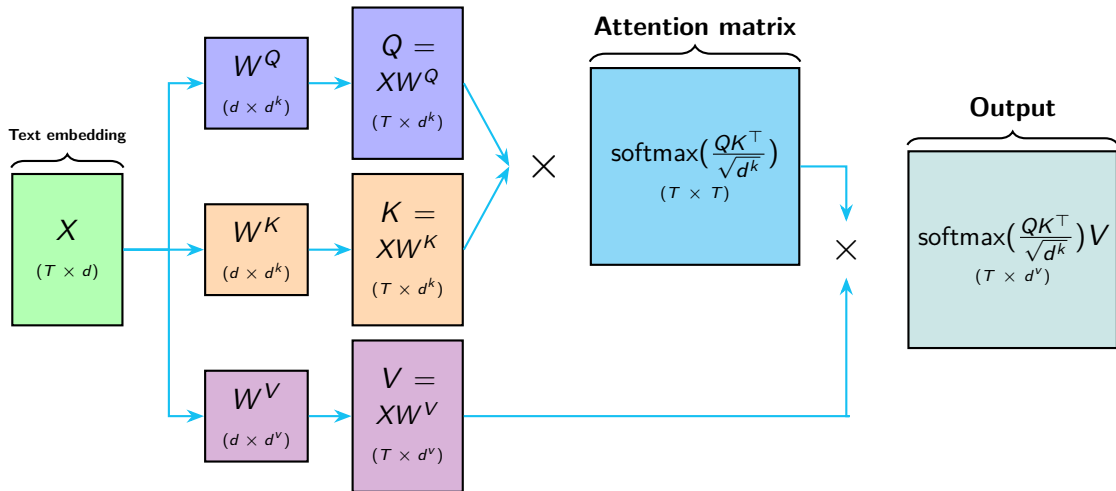
*The **company** reported strong cash flows . . . however, due to regulatory uncertainty . . .*

→ “I want to search the most relevant words *in this sentence* to **company**.”

- ▶ **Query:** the word “company”
- ▶ **Keys:** [“The”, “company”, “reported”, . . . , “uncertainty”]
- ▶ **Values:** [“The”, “company”, “reported”, . . . , “uncertainty”]

The queries, keys, and values all come from the **same text** ⇒ **self**-attention

The architecture



Remarks

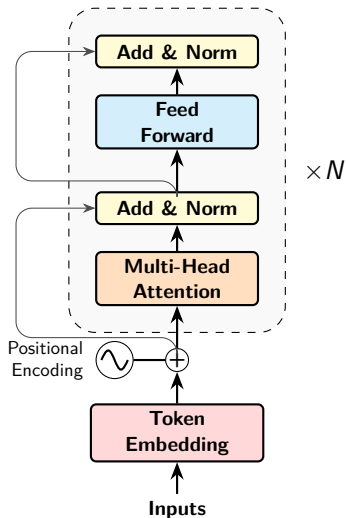
- ▶ **Text embedding = Token embedding + Positional embedding**
- ▶ In practice, we use many *small* self-attention modules in parallel and concatenate their outputs → **multi-head self-attention**
- ▶ Self-attention transforms a sequential modeling problem into matrix multiplications
 - ▶ no recurrent computation → no vanishing gradient problem
 - ▶ highly parallelizable across words

Transformer

Transformer - An application of self-attention

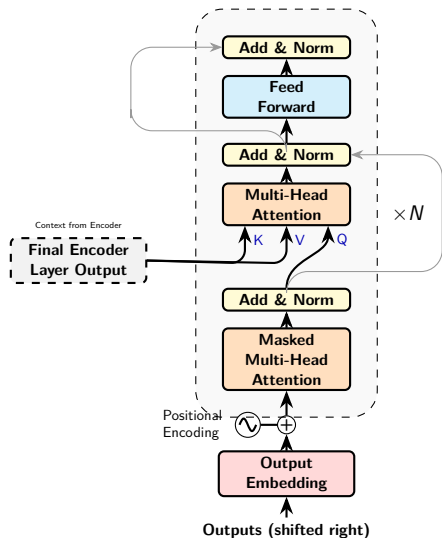
- ▶ **Transformer** is an **encoder–decoder** architecture
 - ▶ **Encoder:** transforms an input text into contextual embeddings
 - ▶ **Decoder:** generates an output text using
 - ▶ the encoder embeddings (*cross-attention*)
 - ▶ the previously generated tokens (*masked self-attention*)

The Transformer encoder



- **Multi-Head Attention:** the self-attention mechanism
- **Add & Norm:** $\text{LayerNorm}(x + \text{Self-Attention}(x))$
- **Feed Forward:** 2-layer linear projection, $\max(0, xW_1 + b_1)W_2 + b_2$

The Transformer decoder



- **Intuition:** the decoder predicts the next token by combining
 - information from previously generated tokens
 - relevant context from the encoder outputs
- **Masked Multi-Head Self-Attention:** prevents a token from attending to future tokens (implemented by masking attention scores with $-\infty$)
- **Cross-Attention:**
 - queries come from the decoder
 - keys and values come from the encoder
- **Autoregressive:**
 - prevents a word from looking at future words

What you need to remember

- ▶ **Self-attention** transforms static token embeddings into **contextualized embeddings**
 - ▶ with a similar goal as ELMo
 - ▶ different mechanism: matrix multiplication instead of recurrence
- ▶ **Transformers** are architectures built around self-attention
 - ▶ stack self-attention with feed-forward layers
 - ▶ add residual connections and normalization
- ▶ Many modern language models use *parts* of the Transformer architecture
 - ▶ encoder-only (e.g., BERT)
 - ▶ decoder-only (e.g., GPT)

Language models

From Transformers to language models

- ▶ A **language model** (i.e., a neural network with parameters Θ) assigns probabilities to a sequence of tokens by modeling the conditional distribution of each token given its context

$$p(w_1, w_2, \dots, w_T | \Theta) = \prod_{t=1}^T p(w_t | w_1, \dots, w_{t-1}, \Theta)$$

- ▶ Transformer-based language models use
 - ▶ self-attention to build contextualized token representations
 - ▶ deep stacks of Transformer layers to capture complex patterns

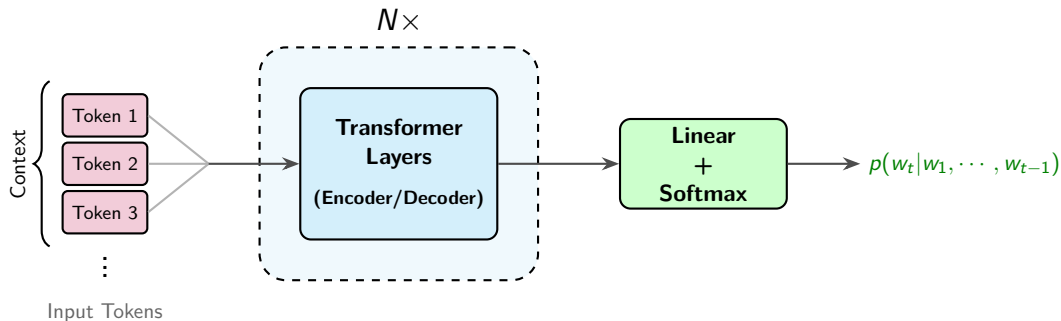
From Transformers to language models

- ▶ A **language model** (i.e., a neural network with parameters Θ) assigns probabilities to a sequence of tokens by modeling the conditional distribution of each token given its context

$$p(w_1, w_2, \dots, w_T | \Theta) = \prod_{t=1}^T p(w_t | w_1, \dots, w_{t-1}, \Theta)$$

- ▶ Transformer-based language models use
 - ▶ self-attention to build contextualized token representations
 - ▶ deep stacks of Transformer layers to capture complex patterns
- ▶ Different architectures lead to different language models:
 - ▶ **Decoder-only** → GPT-style models
 - ▶ **Encoder-only** → BERT-style models
 - ▶ **Encoder-decoder** → sequence-to-sequence models

General language model architecture



Critical question: *How can we train a language model?*

Generative Pre-Training models - GPT

- ▶ Introduced by Radford et al. [2018] at OpenAI

Generative Pre-Training models - GPT

- ▶ Introduced by Radford et al. [2018] at OpenAI
- ▶ Core training idea: learn a language model that predicts each token using all preceding tokens (autoregressive modeling)

Generative Pre-Training models - GPT

- ▶ Introduced by Radford et al. [2018] at OpenAI
- ▶ Core training idea: learn a language model that predicts each token using all preceding tokens (autoregressive modeling)
- ▶ This mirrors the text generation process → hence the name **Generative** Pre-training (GPT)

Generative Pre-Training models - GPT

- ▶ Introduced by Radford et al. [2018] at OpenAI
- ▶ Core training idea: learn a language model that predicts each token using all preceding tokens (autoregressive modeling)
- ▶ This mirrors the text generation process → hence the name **Generative** Pre-training (GPT)
- ▶ Given: ***the company reports high earnings***, GPT is trained by answering
 - ▶ Given “the”, can you (GPT) predict the next word (i.e., “company”), then
 - ▶ Given “the company”, can you predict the next word (i.e., reports), then
 - ▶ Given “the company reports”, can you predict the next word (i.e., high), and so on

Generative Pre-Training models - GPT

- ▶ Introduced by Radford et al. [2018] at OpenAI
- ▶ Core training idea: learn a language model that predicts each token using all preceding tokens (autoregressive modeling)
- ▶ This mirrors the text generation process → hence the name **Generative** Pre-training (GPT)
- ▶ Given: ***the company reports high earnings***, GPT is trained by answering
 - ▶ Given “the”, can you (GPT) predict the next word (i.e., “company”), then
 - ▶ Given “the company”, can you predict the next word (i.e., reports), then
 - ▶ Given “the company reports”, can you predict the next word (i.e., high), and so on
- ▶ Formally, estimate parameters Θ to maximize $p(w_t \mid w_1, \dots, w_{t-1}; \Theta)$, which is equivalent to maximizing the log-likelihood:

$$\mathcal{L}(\Theta) = \sum_{t=1}^T \log p(w_t \mid w_1, \dots, w_{t-1}; \Theta)$$

Generative Pre-Training models - GPT

- ▶ Introduced by Radford et al. [2018] at OpenAI
- ▶ Core training idea: learn a language model that predicts each token using all preceding tokens (autoregressive modeling)
- ▶ This mirrors the text generation process → hence the name **Generative** Pre-training (GPT)
- ▶ Given: ***the company reports high earnings***, GPT is trained by answering
 - ▶ Given “the”, can you (GPT) predict the next word (i.e., “company”), then
 - ▶ Given “the company”, can you predict the next word (i.e., reports), then
 - ▶ Given “the company reports”, can you predict the next word (i.e., high), and so on
- ▶ Formally, estimate parameters Θ to maximize $p(w_t \mid w_1, \dots, w_{t-1}; \Theta)$, which is equivalent to maximizing the log-likelihood:

$$\mathcal{L}(\Theta) = \sum_{t=1}^T \log p(w_t \mid w_1, \dots, w_{t-1}; \Theta)$$

- ▶ Architecture: decoder-only Transformer (12 Transformer decoder layers)

Bidirectional Encoder Representations from Transformers - BERT

- ▶ Devlin et al. [2019] claim that, in sentence-level tasks, such as sentiment analysis, paying attention to only preceding words is insufficient

Bidirectional Encoder Representations from Transformers - BERT

- ▶ Devlin et al. [2019] claim that, in sentence-level tasks, such as sentiment analysis, paying attention to only preceding words is insufficient
- ▶ Core training idea: train a language model to predict a word *given surrounding words*, not just those before it; hence the name **Bidirectional** ...

Bidirectional Encoder Representations from Transformers - BERT

- ▶ Devlin et al. [2019] claim that, in sentence-level tasks, such as sentiment analysis, paying attention to only preceding words is insufficient
- ▶ Core training idea: train a language model to predict a word *given surrounding words*, not just those before it; hence the name **Bidirectional** ...
- ▶ They introduce the Masked Language Modeling task: randomly mask a subset of tokens and predict them using the remaining tokens

Bidirectional Encoder Representations from Transformers - BERT

- ▶ Devlin et al. [2019] claim that, in sentence-level tasks, such as sentiment analysis, paying attention to only preceding words is insufficient
- ▶ Core training idea: train a language model to predict a word *given surrounding words*, not just those before it; hence the name **Bidirectional** ...
- ▶ They introduce the Masked Language Modeling task: randomly mask a subset of tokens and predict them using the remaining tokens
- ▶ Formally, define $\mathcal{M} \subset \{1, \dots, T\}$ the set of masked positions in a sentence; BERT maximizes

$$\mathcal{L}(\Theta) = \sum_{t \in \mathcal{M}} \log p(w_t | w_{\setminus \mathcal{M}}, \Theta)$$

Bidirectional Encoder Representations from Transformers - BERT

- ▶ Devlin et al. [2019] claim that, in sentence-level tasks, such as sentiment analysis, paying attention to only preceding words is insufficient
- ▶ Core training idea: train a language model to predict a word *given surrounding words*, not just those before it; hence the name **Bidirectional** ...
- ▶ They introduce the Masked Language Modeling task: randomly mask a subset of tokens and predict them using the remaining tokens
- ▶ Formally, define $\mathcal{M} \subset \{1, \dots, T\}$ the set of masked positions in a sentence; BERT maximizes

$$\mathcal{L}(\Theta) = \sum_{t \in \mathcal{M}} \log p(w_t | w_{\setminus \mathcal{M}}, \Theta)$$

- ▶ Architecture: encoder-only Transformer (12 Transformer encoder layers)

Bidirectional Encoder Representations from Transformers - BERT

- ▶ Devlin et al. [2019] claim that, in sentence-level tasks, such as sentiment analysis, paying attention to only preceding words is insufficient
- ▶ Core training idea: train a language model to predict a word *given surrounding words*, not just those before it; hence the name **Bidirectional** ...
- ▶ They introduce the Masked Language Modeling task: randomly mask a subset of tokens and predict them using the remaining tokens
- ▶ Formally, define $\mathcal{M} \subset \{1, \dots, T\}$ the set of masked positions in a sentence; BERT maximizes

$$\mathcal{L}(\Theta) = \sum_{t \in \mathcal{M}} \log p(w_t | w_{\setminus \mathcal{M}}, \Theta)$$

- ▶ Architecture: encoder-only Transformer (12 Transformer encoder layers)
- ▶ Because BERT conditions on both past and future tokens during training, it is not naturally suited for **text representation**, not autoregressive text generation

Tokenization

Tokenization

Given a sentence: “Let’s do tokenization!”. There are several tokenization methods:

1. [Let’s, do, tokenization!] (*word-based; the simplest*)
 - ▶ Punctuation remains attached to words
 - ▶ Vocabulary size depends heavily on forms (e.g., “tokenization” vs. “tokenization!”)

Tokenization

Given a sentence: “Let’s do tokenization!”. There are several tokenization methods:

1. [Let’s, do, tokenization!] (*word-based; the simplest*)
 - ▶ Punctuation remains attached to words
 - ▶ Vocabulary size depends heavily on forms (e.g., “tokenization” vs. “tokenization!”)
2. [Let, ', s, do, tokenization, !] (*word + punctuation tokens*)
 - ▶ Leads to a **very large vocabulary** due to surface variations
 - ▶ Example: “!”, “!!!”, and “?!?!” are treated as distinct tokens
 - ▶ Large vocabularies increase memory usage and computational cost; e.g., [Transformer-XL](#) used large fixed vocabularies (~267k tokens)

Tokenization

Given a sentence: “Let’s do tokenization!”. There are several tokenization methods:

1. [Let’s, do, tokenization!] (*word-based; the simplest*)
 - ▶ Punctuation remains attached to words
 - ▶ Vocabulary size depends heavily on forms (e.g., “tokenization” vs. “tokenization!”)
2. [Let, ', s, do, tokenization, !] (*word + punctuation tokens*)
 - ▶ Leads to a **very large vocabulary** due to surface variations
 - ▶ Example: “!”, “!!!”, and “?!?! ” are treated as distinct tokens
 - ▶ Large vocabularies increase memory usage and computational cost; e.g., [Transformer-XL](#) used large fixed vocabularies (~267k tokens)
3. [L, e, t, ', s, d, o, t, o, k, e, n, i, z, a, t, i, o, n, !] (*character-based; most granular*)
 - ▶ Very small vocabulary (e.g., ~100–300 symbols)
 - ▶ Eliminates out-of-vocabulary (OOV) problems
 - ▶ However, tokens become less semantically meaningful
 - ▶ Sequences become much longer → harder long-range dependency modeling

Tokenization

Given a sentence: “Let’s do tokenization!”. There are several tokenization methods:

1. [Let’s, do, tokenization!] (*word-based; the simplest*)
 - ▶ Punctuation remains attached to words
 - ▶ Vocabulary size depends heavily on forms (e.g., “tokenization” vs. “tokenization!”)
2. [Let, ', s, do, tokenization, !] (*word + punctuation tokens*)
 - ▶ Leads to a **very large vocabulary** due to surface variations
 - ▶ Example: “!”, “!!!”, and “?!?! ” are treated as distinct tokens
 - ▶ Large vocabularies increase memory usage and computational cost; e.g., [Transformer-XL](#) used large fixed vocabularies (~267k tokens)
3. [L, e, t, ', s, d, o, t, o, k, e, n, i, z, a, t, i, o, n, !] (*character-based; most granular*)
 - ▶ Very small vocabulary (e.g., ~100–300 symbols)
 - ▶ Eliminates out-of-vocabulary (OOV) problems
 - ▶ However, tokens become less semantically meaningful
 - ▶ Sequences become much longer → harder long-range dependency modeling

A method that keeps vocabulary manageable while preserving semantic structure is wanted!

Subword tokenization

Core idea of subword tokenization

Consider four words: “normally”, “beautifully”, “normal”, “beautiful”

- ▶ Standard word-based tokenization produces a vocabulary of 4 distinct tokens

Subword tokenization

Core idea of subword tokenization

Consider four words: “normally”, “beautifully”, “normal”, “beautiful”

- ▶ Standard word-based tokenization produces a vocabulary of 4 distinct tokens
- ▶ However, these words share common roots and suffixes

Subword tokenization

Core idea of subword tokenization

Consider four words: “normally”, “beautifully”, “normal”, “beautiful”

- ▶ Standard word-based tokenization produces a vocabulary of 4 distinct tokens
- ▶ However, these words share common roots and suffixes
- ▶ A subword tokenizer can instead construct a smaller vocabulary:

[normal, beautiful, ly]

Subword tokenization

Core idea of subword tokenization

Consider four words: “normally”, “beautifully”, “normal”, “beautiful”

- ▶ Standard word-based tokenization produces a vocabulary of 4 distinct tokens
- ▶ However, these words share common roots and suffixes
- ▶ A subword tokenizer can instead construct a smaller vocabulary:

[normal, beautiful, ly]

- ▶ The original words are then represented as:
 - ▶ normally = normal + ly
 - ▶ beautifully = beautiful + ly

Subword tokenization

Core idea of subword tokenization

Consider four words: “normally”, “beautifully”, “normal”, “beautiful”

- ▶ Standard word-based tokenization produces a vocabulary of 4 distinct tokens
- ▶ However, these words share common roots and suffixes
- ▶ A subword tokenizer can instead construct a smaller vocabulary:

[normal, beautiful, ly]

- ▶ The original words are then represented as:
 - ▶ normally = normal + ly
 - ▶ beautifully = beautiful + ly
- ▶ This method is particularly useful for agglutinative languages such as German or Turkish, where words are formed by concatenating subwords together

Subword tokenization

Core idea of subword tokenization

Consider four words: “normally”, “beautifully”, “normal”, “beautiful”

- ▶ Standard word-based tokenization produces a vocabulary of 4 distinct tokens
- ▶ However, these words share common roots and suffixes
- ▶ A subword tokenizer can instead construct a smaller vocabulary:

[normal, beautiful, ly]

- ▶ The original words are then represented as:
 - ▶ normally = normal + ly
 - ▶ beautifully = beautiful + ly
- ▶ This method is particularly useful for agglutinative languages such as German or Turkish, where words are formed by concatenating subwords together
- ▶ All subwords and words now are referred to as **tokens**

How do we perform subword tokenization?

- ▶ Word-level tokenization is straightforward, e.g., `text.split()` in Python

How do we perform subword tokenization?

- ▶ Word-level tokenization is straightforward, e.g., `text.split()` in Python
- ▶ Subword tokenization is more involved: The vocabulary is *learned from data*

How do we perform subword tokenization?

- ▶ Word-level tokenization is straightforward, e.g., `text.split()` in Python
- ▶ Subword tokenization is more involved: The vocabulary is *learned from data*
- ▶ Two widely used approaches:
 - ▶ Byte-Pair Encoding (BPE) [Sennrich et al., 2016]—used in GPT models
 - ▶ WordPiece [Schuster and Nakajima, 2012]—used in BERT

How do we perform subword tokenization?

- ▶ Word-level tokenization is straightforward, e.g., `text.split()` in Python
- ▶ Subword tokenization is more involved: The vocabulary is *learned from data*
- ▶ Two widely used approaches:
 - ▶ Byte-Pair Encoding (BPE) [Sennrich et al., 2016]—used in GPT models
 - ▶ WordPiece [Schuster and Nakajima, 2012]—used in BERT
- ▶ Core idea:
 - ▶ Start from a base vocabulary of individual characters
 - ▶ Iteratively merge character sequences into subword units
 - ▶ Merge criterion:
 - ▶ BPE: frequency-based
 - ▶ WordPiece: likelihood/probability-based
 - ▶ Stop when a predefined criterion is met; e.g., OpenAI **GPT** has a vocabulary size of 40,478, consisting of 478 base characters and 40,000 merge operations

Example of BPE (taken from HuggingFace)

Example corpus

pun pun pun pun pun pun pun pun pun pun pun pun
hug hug hug hug hug hug hug hug hug hug hug
pug pug pug pug pug
hugs hugs hugs hugs hugs
bun bun bun bun

1. Extract the unique words with their frequencies:
[("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)]
2. The base vocabulary (unique characters) is ["b", "g", "h", "n", "p", "s", "u"]

Example of BPE - continued

3. Split each word into characters in the base vocabulary: (“h” “u” “g”, 10), (“p” “u” “g”, 5), (“p” “u” “n”, 12), (“b” “u” “n”, 4), (“h” “u” “g” “s”, 5)
4. First merge step:
 - ▶ The most frequent adjacent pair is “ug” (20 occurrences)
 - ▶ After merging: (“h” “ug”, 10), (“p” “ug”, 5), (“p” “u” “n”, 12), (“b” “u” “n”, 4), (“h” “ug” “s”, 5)
5. Second merge step:
 - ▶ The most frequent pair is “un” (16 occurrences)
 - ▶ After merging: (“h” “ug”, 10), (“p” “ug”, 5), (“p” “un”, 12), (“b” “un”, 4), (“h” “ug” “s”, 5)
6. Suppose we **stop after two merge operations**. The resulting subword units, together with the base vocabulary, form the final vocabulary.
7. Special tokens are typically added to the vocabulary, such as: “[PAD]”, “[UNK]”, “[CLS]”, “[SEP]”, “[MASK]”
 - ▶ If the tokenizer encounters a new word not in the vocabulary, e.g., “mug”, it is split into: (“[UNK]”, “ug”) since “m” is missing from the final vocabulary

About WordPiece

- ▶ Very similar to BPE; the key difference is that merges are based on **likelihood/probability** rather than raw frequency
- ▶ For each candidate pair, WordPiece computes a score:

$$\text{score} = \frac{\text{freq_of_pair}}{\text{freq_of_first_element} \times \text{freq_of_second_element}}$$

- ▶ Intuition:
 - ▶ freq_of_pair: how much information we **obtain** by merging
 - ▶ freq_of_first_element × freq_of_second_element: how much information we **lose** from the merge
- ▶ Only pairs with a favorable score are merged — i.e., we merge if it is **worth it** and do not lose significant information

Pros & Cons of language models (LMs)

► Pros:

- State-of-the-art language modeling techniques that overcome limitations of previous methods, such as lack of contextualization, vanishing gradients, and sequential modeling constraints
- Capture complex linguistic structures, including nuanced expressions in financial text data
- Linguistic features extracted from LMs (sentiment, uncertainty, embeddings, etc.) are generally more accurate and reliable than those from traditional methods

Pros & Cons of language models (LMs)

► Pros:

- State-of-the-art language modeling techniques that overcome limitations of previous methods, such as lack of contextualization, vanishing gradients, and sequential modeling constraints
- Capture complex linguistic structures, including nuanced expressions in financial text data
- Linguistic features extracted from LMs (sentiment, uncertainty, embeddings, etc.) are generally more accurate and reliable than those from traditional methods

► Cons:

- Require substantial computational resources
- Lack of transparency: difficult to interpret decision-making (“black box”)
- **Challenging to handle very long texts efficiently**

Self-attention - Bittersweetness of language models (LMs)

- ▶ Self-attention is, at the same time, the core and the bottleneck of every LM
- ▶ Particularly, the attention matrix is square: $T \times T$, where T is the document length (number of tokens)
- ▶ Example:
 - ▶ A document with 10 tokens produces an attention matrix with 100 elements
 - ▶ A document with 20 tokens produces an attention matrix with **400** elements
→ Doubling the document length requires roughly 4 times more resources
- ▶ This is referred to as **quadratic complexity** in self-attention
- ▶ In finance, this is a huge problem because financial texts are usually very long!
 - ▶ An average 10-K document (annual financial statements of US firms) has approx. 55,000 words or 70,000 tokens
 - ▶ An LM (such as BERT) using single-precision floating-point format (i.e., 32 bits or four bytes) requires $4 \times 55,000^2 \times 12 = 145.2$ gigabytes to process only one 10-K!

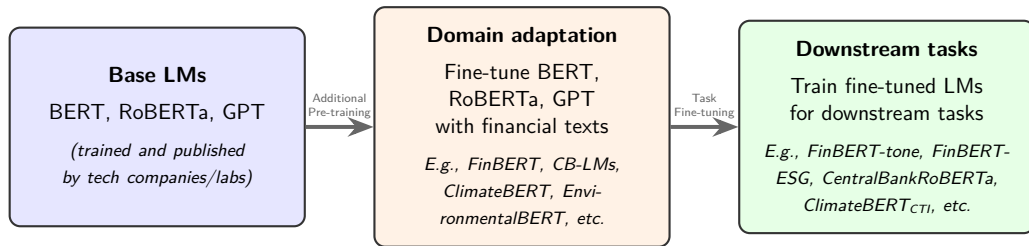
Language models in financial research

Financial language models

Various LMs have been introduced for different types of financial documents

- ▶ [FinBERT](#) and [FinBERT-tone](#) [Huang et al., 2023] for sentiment analysis of analysts' reports
- ▶ [CentralBankRoBERTa-agent-classifier](#), [CentralBankRoBERTa-sentiment-classifier](#) [Pfeifer and Marohl, 2023], [CB-LMs](#) for central bank communication analysis
- ▶ [ClimateBERT](#) [Kölbel et al., 2024, Webersinke et al., 2022] and [ClimateBERT_{CTI}](#) [Bingler et al., 2024] for corporate climate reports
- ▶ [EnvironmentalBERT](#) [Schimanski et al., 2024] and [FinBERT-ESG](#) [Huang et al., 2023] for ESG-related (environmental, social, and governance) texts
- ▶ [PatentSBERTa](#) [Bekamiri et al., 2024] for patent classification

Domain adaptation pipeline



- ▶ Training base LMs are costly. For instance, training BERT-base (109M parameters) on a 250-billion-token corpus using one A100 80GB SXM ($\approx$ \$18,000) needs 15.2 days (estimated based on Brown et al. [2020])
- ▶ Financial texts include financial news, central bank communication, earnings call transcripts, financial statements, etc.
- ▶ Downstream tasks typically require labeled data

Challenges of LMs in finance

- ▶ **Limited labeled data for downstream tasks**
 - ▶ Financial annotations (e.g., risk, tone, ESG) require domain expertise
 - ▶ Small sample sizes relative to model complexity
- ▶ **Long and complex documents**
 - ▶ 10-K filings, annual reports, regulatory documents
 - ▶ Transformer models have quadratic complexity in document length
- ▶ **Domain shift and regime changes**
 - ▶ Financial language evolves (words like NFTs, DeFi, etc., have just appeared recently)
 - ▶ Models trained on past data may not generalize
- ▶ **Integration of text and numerical data**
 - ▶ Financial decisions depend on accounting numbers and time series
 - ▶ Standard LMs are weak at numerical reasoning
- ▶ **Interpretability and economic meaning**
 - ▶ Black-box predictions are difficult to justify
 - ▶ Finance research often requires structural interpretation

Coming next

Coming next

- ▶ Hands-on practice
 - ▶ Anatomy of FinBERT's tokenizer, configuration, and model architecture
 - ▶ Sentiment analysis with FinBERT
 - ▶ [Notebook](#)/[Solution](#)
- ▶ Next lecture
 - ▶ Large Language Models

References I

- Hamid Bekamiri, Daniel S Hain, and Roman Jurowetzki. PatentSBERTa: A deep NLP based hybrid model for patent distance and classification using augmented SBERT. *Technological Forecasting and Social Change*, 206:123536, 2024.
- Julia Anna Bingler, Mathias Kraus, Markus Leippold, and Nicolas Webersinke. How cheap talk in climate disclosures relates to climate initiatives, corporate emissions, and reputation risk. *Journal of Banking & Finance*, 164:107191, 2024.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186. Association for Computational Linguistics, June 2019.
- Eugene F Fama. Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2):383–417, 1970.
- Allen H Huang, Hui Wang, and Yi Yang. FinBERT: A large language model for extracting information from financial text. *Contemporary Accounting Research*, 40(2):806–841, 2023.
- Julian F Kölbel, Markus Leippold, Jordy Rillaerts, and Qian Wang. Ask BERT: How regulatory disclosure of transition and physical climate risks affects the CDS term structure. *Journal of Financial Econometrics*, 22(1):30–69, 2024.
- Moritz Pfeifer and Vincent P Marohl. CentralBankRoBERTa: A fine-tuned large language model for central bank communications. *The Journal of Finance and Data Science*, 9:100114, 2023.
- Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.
- Tobias Schimanski, Andrin Reding, Nico Reding, Julia Bingler, Mathias Kraus, and Markus Leippold. Bridging the gap in ESG measurement: Using NLP to quantify environmental, social, and governance communication. *Finance Research Letters*, 61:104979, 2024. ISSN 1544-6123.
- Mike Schuster and Kaisuke Nakajima. Japanese and Korean voice search. In *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5149–5152. IEEE, 2012.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (volume 1: long papers)*, pages 1715–1725, 2016.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- Nicolas Webersinke, Mathias Kraus, Julia Bingler, and Markus Leippold. ClimateBERT: A Pretrained Language Model for Climate-Related Text. In *Proceedings of AAAI 2022 Fall Symposium: The Role of AI in Responding to Climate Challenges*, 2022.